🚀 **Ellstack Amigo AI Stack:** Ultra-Fast Multi-Format RAG API Documentation

Overview

Ellstack Amigo is a comprehensive AI stack designed to simplify the integration of local AI models with WhatsApp in a scalable and production-ready way. It features an advanced automatic context management system, built-in model auto-training capabilities, and a powerful, reusable API that enables rapid development of local AI-driven solutions.

Out of the box, Ellstack Amigo includes seamless WhatsApp integration, allowing developers to deploy conversational AI solutions immediately without complex setup.

Ellstack AI chatbot Amigo app

This app allows you to quickly set up RAG data visually and connect your WhatsApp to the bot, enabling it to start chatting with customers.

To view the complete Ellstack AI Chatbot Amigo app user guide, click **here**.

Amigo AI Stack includes:

- 🗔 Amigo setup windows APP
- 🧠 Local AI model (llama 3.2 3B default)
- 🤖 Ellstack Model API for accessing the AI Local Model
- ⚡ Ellstack API with this feature:
    - Advanced context manager with multi-format support and multi-retrieval algorithms
    - Seamless integration with local AI models
    - High-performance in-memory caching (RAM-based)
    - Vector database capabilities for efficient semantic search
    - Automated model training and continuous improvement

- 📱 Local WhatsApp integration using Baileys, enhanced with a wide range of additional features:
    - Automatic voice message transcription
    - AI-powered message matching to identify the same customer across devices (e.g., mobile and WhatsApp Web generating different numbers)
    - AI-based output cleanup for local models to reduce or correct hallucinations

AMIGO AI STACK API SPECIFICATION

Base URL

http://localhost:8964

📌 Endpoints

1. 🧠 Ask Question (Main Endpoint)

POST /ask

Main entry point for: RAG + Advanced context manager + LLM.

Request Body

```
{
  "question": "Do you have 235/40R18 tires?",
  "session_id": "145415458614",
  "system": "You a tire shop representative",
  "temperature": 0.1,
  "top_p": 0.1,
  "top_k": 2,
  "max_token": 128,
  "cache_max_use": 3
}
```

Parameters

Field	Type	Required	Description
`question`	string	✅	User input question for AI model
`session_id`	string	❌	Session id for saving chat conversation history
`system`	string	❌	Override system prompt
`temperature`	float	❌	LLM randomness (default: 0.1)
`top_p`	float	❌	Nucleus sampling (default: 0.1)
`top_k`	int	❌	Token selection (default: 2)
`max_token`	int	❌	Max tokens generated
`cache_max_use`	int	❌	Max RAM cache use for same question

Response

```
{
  "answer": "Yes, we have 235/40R18 tires available
for $120."
}
```

Execution Flow (IMPORTANT)

1. **RAM Cache (FASTEST)**
    a. Exact normalized question match
    b. Returns a cached response if the question's usage count is below the configured cache_max_use limit.

2. **Persistent Vector Search**
    a. Semantic FAQ match
    b. Threshold: `0.55`

3. **RAG + Memory Vector Search+ AI MODEL LLM**
    a. Hybrid scoring:
        i. Semantic similarity
        ii. Keyword match boost
    b. Context sent to LLM

Notes

- Context is capped to **12,000 characters**
- Chat history limit: **10 messages** (if used **session_id**)

- Session memory stored in RAM Memory for (24h TTL)

Example (cURL)

```
curl -X POST http://localhost:8964/ask \
-H "Content-Type: application/json" \
-d '{
  "question": "Price of 235/40 R18 tires",
  "session_id": "abc123"
}'
```

2. 🎙 Audio Transcription

POST /transcribe

Transcribes audio using Whisper.

Request

```
{
  "path": "/path/to/audio/file.wav"
}
```

Response

```
{
  "text": "Hola, necesito información sobre gomas."
}
```

3. 📱 Phone AI Matcher

`POST /phone_ai_matcher`

Matches outbound messages to most similar inbound conversation using embeddings.

Request

```
{
  "phone_number": "18095551234",
  "date_send": "2026-04-13"
}
```

Response

```
{
  "match_phone": "18095559876",
  "similarity": 0.87,
  "destination_message": "Hello, we have 18-inch tires
available.",
  "original_message": " I need 18-inch tires. "
}
```

How It Works

1. Gets last outbound messages for `phone_number`
2. Gets inbound messages from other numbers
3. Computes cosine similarity (SentenceTransformer)

4. Returns best match
5. Stores result in SQLite (`phone_automatic_match`)

⚙ System Configuration

System Environment Variables

Variable	Description
`ELLSTACK_DIR`	Base directory for system

Important installation directory Paths

Path	Description
`/data`	`ellsdb` SQLite DB, Audio files from whatsapp voice notes, Qdrant storage
`/rag_data`	Documents for RAG Can be excel file (recommed), CSV, TXT, PDF file, DOCX
`/bin`	Llama.cpp files / win64 service programs
`/bin/Models`	File: ellstack_aimodel.gguf (llama 3.2 3B).

	You can download any llama.cpp local model on this folder, and just rename it ellstack_aimodel.gguf
`/bin/service`	AI local Model API Service
`/whatsapp_server`	Whatsapp Baileys with customized API integrated with Ellstack ai stack

Windows Services

Service name	Description
`ellStackAIApi`	`Ellstack API and Services (Context managment system, AI Respond Flow control, etc)`
`ellStack AI Model`	Local AI Model direct API
`ellStack AsyncWSSend`	Sync service and dispatcher for Whatsapp send message and AI Queue handling.
`ellStackWhatsapp_serveri`	Whatsapp server (baileys + custom integrations / routines)

🧠 RAG System Details

In order to use RAG, just place on ellstackdir**\rag_data** folder the files that you want to use for context. Take into account that the stack will automatically classify the documents. If detected Question /Answer format, it will be upload directly to RAM Memory cache for fast responding. All RAG files are uploaded to RAM Vector database in data chunks.

Supported Formats

- CSV
- Excel (.xlsx, .xls)
- PDF
- DOCX
- TXT

Processing Pipeline

1. Load documents
2. Normalize & clean
3. Chunk large documents
4. Generate embeddings (`all-MiniLM-L6-v2`)
5. Store in RAM Vector Database

Retrieval Strategy

Hybrid ranking:

Sentence score match (confidence thresholds (high, medium, low) ,semantic similarity + custom algorithm)

🧠 RAM Cache Usage

Keys

Key Pattern	Description
`qa_cache:*`	Cached answers
`session_ctx:*`	Context memory
`chat:*`	Chat history
`reply_count:*`	Rate limiting

Cache Example

```
{
  "answer": "We have tires available.",
  "uses": 1
}
```

🧠 Direct LLM API Configuration

Endpoint

http://localhost:10064/v1/chat/completions

Default Model:

llama3.2:3b

Payload Example

```
{
  "model": "llama3.2:3b",
  "temperature": 0.1,
  "top-p": 0.1,
  "top-k": 2,
  "n-predict": 128,
  "messages": [
    {"role": "system", "content": "..."},
    {"role": "user", "content": "question"}
  ]
}
```

⚠️ Important for ellstack Amigo AI Model Behaviors

Strict Answer Rules

- ❌ No hallucination
- ❌ No inferred products
- ✅ Only answer from context
- ✅ Preserve exact numeric values
- ✅ Same language as user

Automatic AI Replays Limiting

Sometimes you need to operate in a mixed environment where both humans and AI bots respond to customers via WhatsApp. This feature allows you to limit the number of automatic AI replies per customer within a 24-hour period.

SQLLite table: local_setting, param: AUTOMATIC_REPLIES_PER_CUSTOMER

If automatic_replies_per_customer is set to 0, the Amigo AI agent will respond to all customer messages.

Fallback Logic

```
RAM CACHE→ Persistent Vector Database → RAM Vector Database + LLM
```

🧪 Ellstack AI API Debug Mode

Run with cmd:

```
ellstackdir\ellstack_api.exe -debug
```

Outputs:

- Context used
- Messages sent to LLM
- Parameters

🧩 Example Full Flow

Input

```
{
  "question": "tienen gomas aro 18 235 40",
  "session_id": "user1"
}
```

System Does

1. Normalize question
2. Check RAM memory cache (quick FAQS)
3. Search and retrieve context Ram Vector database
4. Merge with session context
5. Send to LLM

6. Cache AI model result on RAM memory (for fast retrieval in case of same question asked again)